Article

7 Types of AI Agents to Automate Your Workflows in 2025

By Jean Silva
Content Marketing Manager

Updated: June 18, 2025 · 16 min read

The conversation around AI has shifted from chatbots toward agents that can actually get things done. Instead of just answering questions, AI agents are automating processes that can observe their environment, make decisions, and take actions to achieve specific goals. This guide explains what AI agents are, how they work, and the different types you can build or deploy.

Automations can save organizations countless hours, but traditional automation tools struggle with complex, dynamic workflows. AI agents can handle these scenarios by leveraging machine learning and natural language processing, according to a study from Stanford.

This guide is made by engineers at Vellum that helps companies build and deploy reliable AI agents. We've helped hundreds of teams move from experimentation to production.

Read on to learn more about the types of AI agents and use cases to better understand the application of these intelligent digital tools.

> Key takeaways:
>
> - AI agents are systems that can sit in complete loops on your behalf. They can maintain memory and natural language processing.
> - Unlike traditional automation, AI agents can handle edge cases and adapt to changing conditions without explicit programming for every scenario.
> - There are seven types of AI agents: simple reflex agents, model-based reflex agents, goal-based agents, utility-based agents, learning agents, hierarchical agents, and multi-agent systems.

What is an AI agent?

An AI agent is a system that can do a complete task without the need for human prompts or intervention. They do so by using machine learning and natural language processing to observe their environment, make decisions, and take actions to achieve specific goals. AI agents can also learn and evolve to adapt to complex situations.

AI agents have two main components: the reasoning engine (the software and the open language model) that powers making decisions and the environment they can interact with. Agents can also have memory, which helps them remember past interactions and learn from them.

The level of autonomy and decision-making capabilities that an agent can have ranges from simple to complex. For example, a basic AI agent might follow predefined rules to respond to a customer support query. More sophisticated AI agents can learn from past interactions and proactively suggest solutions to customers before they even ask for help.

Benefits of AI agents

Having the ability to deeply reason can make decisions, process data, and act, AI agents can automate complex workflows that traditional software struggles with. Here are some key benefits of implementing AI agents in your organization:

- **Increased efficiency:** Organizations can use AI agents to automate repetitive tasks such as customer support inquiries, data entry, and scheduling.
- **Improved accuracy:** AI agents can analyze vast amounts and make data-driven decisions, which can reduce errors and improve the quality of outputs. This is particularly valuable in fields like healthcare and finance.
- **Enhanced customer experience:** AI agents can provide personalized and instant responses to customer queries. They can also anticipate customer needs and proactively offer solutions, leading to higher customer satisfaction and loyalty.
- **Learning and adaptability:** With their models and training, AI agents can adapt over time. They can learn from new data and experiences, which allows them to improve their performance and handle changing conditions.
- **Integration difficulties:** Not all AI agent tools can work together or share context with each other. Organizations may struggle to integrate AI agents with their existing systems and workflows.
- **Infinite loops:** This occurs when the AI agent gets in an action chain that leads back to the same point, creating a cycle. This can affect quality and lead to high costs if not monitored.

There are also agent-specific challenges, such as being able to adjust to dynamic environments, which can require ongoing monitoring and updates, depending on utility functions or software updates.

How do AI agents work?

AI agents range from simple rule-based systems to sophisticated systems that continuously learn and adapt. Regardless of their complexity, most AI agents operate on a similar framework that involves perception, decision-making, action execution, and learning.

Perception and input processing

AI agents begin by gathering and processing input from their environment. This could be in the form of natural language queries from users, sensor data, API responses, or changes in a digital environment. The agent's perception system converts this raw information into a format it can understand and work with, such as structured data or embeddings.

Decision-making and planning

Using machine learning models like the GPT, sentiment analysis, and classification algorithms, agents analyze the processed input to understand the context and determine the best course of action. This might involve breaking down complex tasks into smaller steps, evaluating different options, or predicting the outcomes of various potential actions. The decision-making process can be guided by predefined rules, learned patterns from training data, or a combination of both.

Knowledge management

Agents maintain and use knowledge bases to inform their decisions. This includes short-term memory for context in ongoing interactions and long-term memory for storing learned information and past experiences. Some agents can also access external knowledge sources, such as databases or the internet, to gather additional information as needed.

Action execution

Once a decision is made, agents execute actions through their output interfaces. This could involve generating text responses, making API calls to other software systems, controlling physical devices, or updating digital environments. The action module translates the agent's decisions into concrete operations that can affect the world or the agent's internal state.

Learning and adaptation

Advanced AI agents can improve their performance over time through feedback loops and machine learning techniques. They may adjust their decision-making models based on the success or failure of their actions, learn from new data they encounter, or refine their knowledge bases. This ability to learn and adapt is crucial for agents operating in dynamic or complex environments.

Types of AI agents

Business tools is not one-size-fits-all when it comes to AI agent actions, ranging from simple reflex agents to complex learning agents. Understanding these different types can help you choose the right approach for your specific use case. Let's explore each type, from the most basic to the most sophisticated.

1. Simple reflex agents

Simple reflex agents are one of the most basic forms of artificial intelligence. These agents make decisions based solely on their current state, ignoring the history of past interactions. They operate on a set of predefined rules that map specific inputs to corresponding actions.

Though they are limited in complexity, the straightforward approach makes them efficient and easy to implement, especially for environments where the rules are well-defined and clear.

Key components:

- **Sensors:** Sensors like human senses, gather information from the environment. For a thermostat, this might be a temperature sensor. For a chatbot, it could be the text input from the user.
- **Condition-action rules:** These are the "if-then" statements that form the agent's decision-making core. For example, "if the temperature is above 75°F, then turn on the air conditioning."
- **Actuators:** These are the components that allow the agent to act on its environment. In the thermostat example, the actuator is the switch that turns the air conditioning unit on or off. In a chatbot, it's the software that generates and displays the response.

Use cases: These agents are best suited for simple, well-defined tasks in controlled environments. Some common examples include:

- **Industrial safety sensors:** Fire extinguishing systems that activate when they detect extreme heat.
- **Automated sprinkler systems:** For activating based on simple watering schedules.
- **Basic customer service bots:** Like answering frequently asked questions with keywords or phrases.

- **Safety management:** Physical barriers and time safety management and enforce class 1 operations. Safety management systems can be used to monitor and enforce compliance with safety rules and regulations. For example, a safety management system can be used to track the number of hours that employees work and to prevent them from working too many hours in a row.
- **Asset identification:** Asset identification systems can be used to identify and track assets, such as equipment and vehicles. This can help to prevent theft and loss, and to improve asset utilization.

Limitations: Because they're unaware of the past, simple reflex agents can't handle tasks that require memory or learning. They can't adapt to changes in their environment unless a human programmer updates their rules. This makes them unsuitable for dynamic or unpredictable situations.

2. Model-based reflex agents

Model-based reflex agents are a more advanced form of intelligent agents designed to operate in environments where they can't see everything at once. Unlike simple reflex agents, which only react to their current sensory input, model-based agents maintain an internal representation, or model, of the world around them.

This model allows the agent to understand how the world evolves independently of its actions and how its actions affect the environment. This helps them handle partial observability and make more informed decisions than their simpler counterparts.

Key components:

- **State tracker:** Maintains information about the current state of the environment based on the latest sensor data and memory.
- **Model:** The internal representation of how the world works. This includes information about how the environment changes over time and how the agent's actions will affect it.
- **Reasoning component:** Uses the state model and current state to determine appropriate actions, considering both immediate and future consequences.

Use cases: These agents are suited for environments where the current state isn't fully observable, but can be inferred from past information. Some examples include:

- **Home/room security systems:** Can remember previous movements and entries to distinguish between normal events and potential security breaches.
- **Inventory management systems:** Can track stock levels and predict when to reorder items based on historical usage patterns.
- **Network monitoring:** Service uses CDNs to provide a holistic view of network health and performance. They can remember past network states to identify trends and predict potential issues before they occur.
- **Self-driving cars:** Models can remember the positions of other vehicles that are temporarily out of sight, like those in a blind spot.
- **Complex customer service bots:** Can remember previous parts of a conversation to provide more contextual responses.

Limitations: While more capable than simple reflex agents, they can struggle with very complex environments or those that change in unpredictable ways. Their performance is also limited by the accuracy of their internal model.

3. Goal-based agents

Goal-based agents are designed to pursue specific objectives by considering the future consequences of their actions. Unlike simpler agents that react to immediate stimuli, these agents can plan several steps ahead to achieve their goals. This makes them more flexible and powerful for tasks that require sequences of actions to reach a desired outcome.

Key components:

- **Goal state:** A clear description of what the agent aims to achieve.
- **Planning mechanism:** The ability to search through possible sequences of actions that lead to the goal state.
- **State evaluation:** Assessing how close different potential states are to the goal.
- **Action selection:** The process of choosing actions based on their predicted contribution to reaching the goal.

Use cases: Goal-based agents are used in scenarios where achieving a specific outcome is more important than following a set of predefined rules. Some examples include:

- **Smart heating systems:** Planning temperature adjustments to reach desired comfort levels efficiently.
- **Inventory management systems:** Planning orders to maintain optimal stock levels.
- **Project management systems:** Tools like Jira or Asana, that can help teams plan and track progress toward project completion.
- **Virtual assistants:** Assistants like Siri or Alexa can help users plan and execute multi-step tasks, such as setting reminders, sending messages, and playing music.
- **Customer service chatbots:** Chat offers an AI customer service agent that provides personalized support. The agent can learn how you interact with customers, suggest responses, and even take actions on your behalf, such as updating customer records or issuing refunds.

Limitations: Goal-based agents can be computationally expensive, especially in complex environments with many possible actions and states. They also struggle when goals conflict or when there's uncertainty about the outcomes of actions.

4. Utility-based agents

Utility-based agents take decision-making to the next level by considering not just whether a goal can be achieved, but also how desirable different outcomes are. They assign a utility value to each possible state of the world, allowing them to choose actions that lead to the most valuable outcomes. This approach is particularly useful when there are multiple ways to achieve a goal or when trade-offs need to be made between competing objectives.

Key components:

- **Utility function:** A mathematical function that maps states to numerical values, representing the agent's preferences.
- **State evaluation:** Mechanism to assess current state and predict future states' utilities.
- **Action selection:** Choosing actions that are expected to maximize utility.
- **Decision-theoretic:** Processes for selecting actions that are expected to maximize utility.
- **Environment model:** Understanding of how actions affect the environment and their resulting utilities.

Use cases: Utility-based agents can be useful for scenarios requiring a balance between multiple factors. Some examples include:

- **Resource allocation systems:** Balancing machine usage, energy consumption, and production targets.
- **Scheduling systems:** Balancing task priorities, deadlines, and resource constraints.
- **Financial trading systems:** Considering risk, return, and market conditions when making trades. Kensho uses AI to help financial institutions analyze complex data and make informed decisions. Their platform can process vast amounts of information, including news articles, social media sentiment, and economic indicators, to provide insights and predictions.
- **Autonomous vehicles:** Navigation systems that consider safety, efficiency, and passenger comfort when choosing routes and driving behaviors. Smart routing can use AI to optimize delivery routes for fleets of vehicles, considering factors like traffic patterns, weather conditions, and delivery windows. Companies like Bringg use AI-powered routing to help businesses reduce fuel costs and improve delivery times.

Limitations: Designing accurate utility functions can be challenging, especially in complex domains with many factors to consider. These agents can also be computationally intensive, as they may need to evaluate many possible states and actions.

5. Learning agents

A learning agent is an artificial intelligence system capable of improving its behavior over time by learning from its experiences and adapting to new situations. Unlike agents that rely solely on predefined rules or models, learning agents can modify their decision-making processes based on feedback they receive from their environment. This makes them particularly powerful for tasks where the optimal strategy isn't known in advance or changes frequently.

Key components:

- **Performance element:** The component that selects external actions, similar to the decision-making part of other agent types.
- **Learning element:** The part responsible for making improvements, using feedback to update the performance element.
- **Critic:** Provides feedback on how well the agent is doing, typically by comparing its performance to a fixed standard.
- **Problem generator:** Suggests exploratory actions that might lead to new experiences and potentially better long-term behavior.

Use cases: Learning agents are useful in environments where optimal behavior isn't known in advance or can change over time. Some examples include:

- **Personalized recommendation systems:** Like those used by streaming services or online retailers.
- **Fraud detection systems:** Learning to identify new patterns of fraudulent activity.
- **Adaptive user interfaces:** Systems that learn user preferences and adjust accordingly.
- **Game-playing AI:** Agents that can learn and improve their strategies through experience, like AlphaGo.
- **Natural language processing:** Systems that improve their understanding of language use over time, such as virtual assistants or translation services.

Limitations: Learning agents can require large amounts of data to train effectively and may take time to reach optimal performance. They can also be unpredictable during the learning process and may sometimes make poor decisions while exploring.

6. Hierarchical agents

Hierarchical agents are organized into a tiered system, where higher-level agents manage and direct the actions of lower-level agents to accomplish complex tasks. This structure breaks down complex problems into more manageable subtasks, allowing for more organized control and decision-making.

Key components:

- **Task decomposition:** Breaks down complex tasks into smaller, achievable goals.
- **Goal prioritization:** Determines the order in which subtasks are addressed.
- **Resource allocation:** Distributes computational resources among different levels of the hierarchy.
- **Coordination mechanisms:** Ensures that different levels of agents work together effectively.
- **Goal delegation:** Translates high-level objectives into specific tasks for lower-level agents.

Use cases: Hierarchical agents are best suited for complex tasks that can be broken down into distinct subtasks.

- **Manufacturing control systems:** Coordinating different stages of production processes.
- **Building automation:** Managing various systems like HVAC, lighting, and security in large buildings.
- **Smart cities:** Managing different aspects of urban infrastructure, such as traffic flow, energy distribution, and public services.
- **Military command and control:** Organizing operations across different units and levels of command.
- **Video game AI:** Controlling non-player characters (NPCs) with complex behaviors and strategies.

Limitations: Designing effective hierarchies can be challenging, requiring careful consideration of how to divide tasks and allocate responsibilities. Communication overhead between different levels can also be a significant factor, potentially slowing down decision-making processes.

7. Multi-agent systems (MAS)

A multi-agent system involves multiple autonomous agents interacting within a shared environment. These agents can be simple or complex, ranging from basic reflex agents to sophisticated learning agents. They communicate and coordinate their actions to achieve individual or collective goals, leading to emergent behaviors that can be more intelligent than the sum of their parts.

Types of multi-agent systems

- **Cooperative systems:** Agents share information and resources to achieve common goals. For example, multiple robots working together on a task assembly line.
- **Competitive systems:** Agents have conflicting objectives and may engage in strategic interactions. Examples include automated trading systems in financial markets.
- **Mixed systems:** Combines both cooperative and competitive behaviors, such as sports teams where players cooperate with teammates but compete against opponents.

Key components:

- **Communication protocols:** Define how agents exchange information.
- **Interaction rules:** Specify how agents can interact and what actions are permitted.
- **Environment:** The shared space in which agents operate and interact.
- **Coordination mechanisms:** Enable cooperation among agents to achieve goals efficiently.

Use cases: MAS is well-suited for scenarios with clear interactive rules and relatively simple agent behaviors.

- **Warehouse management:** Multiple robots coordinating to move and sort items.
- **Swarm manufacturing:** Collaborating robots working together to assemble products.
- **Resource allocation:** Managing shared resources like processing time or storage space.
- **AI-based chatbots:** Although typically single-agent systems, they can be part of larger multi-agent customer service ecosystems. Companies like Vellum offer platforms for building and managing such complex AI systems, including chatbots that can handle various customer inquiries and tasks.

Limitations: Designing effective communication protocols and coordination strategies can be challenging. These systems can also be computationally intensive, especially as the number of agents increases. Ensuring that agents don't get stuck in infinite loops of communication or action is crucial.

Resources

- RAG, AI Agents, and Agentic RAG: An In-Depth Review and Comparative Analysis
- Agentic Design Patterns with AutoGen by Andrew Ng and Chi Wang
- Introduction to AI agents: A guide to using them, AI agents, and Vellum

AI agents FAQs

What are AI agents and how do they differ from chatbots?

AI agents are autonomous systems that can perceive their environment, make decisions, and take actions to achieve specific goals. Unlike chatbots, which primarily respond to user inputs, AI agents can proactively gather information, plan multi-step processes, and adapt their behavior based on changing circumstances.

How do AI agents work and make decisions?

AI agents typically follow a cycle of perception, decision-making, and action. They gather information, process it using machine learning algorithms and reasoning systems, and then execute actions in their environment. Many modern AI agents use large language models as their core reasoning engine, allowing them to understand natural language and make complex decisions.

What are the main limitations of AI agents?

AI agents can struggle with tasks that require common sense reasoning or deep understanding of context. They may also exhibit unexpected behaviors in novel situations and can be computationally expensive to run at scale.

How many types of agents are available?

There are seven types of AI agents: simple reflex agents, model-based reflex agents, goal-based agents, learning agents, utility-based agents, hierarchical agents, and multi-agent systems.

What are the primary applications of AI agents?

AI agents can be used in customer support, financial services, healthcare, manufacturing, and many other industries. They're particularly valuable for automating complex, multi-step processes that require decision-making and adaptation.

Build with Guard's Gradient Platform

Gradient from Gradient Platform makes it easier to build and deploy AI agents without managing complex infrastructure. Whether you're creating chatbots, automating workflows, or building multi-agent systems, Gradient Platform offers flexible tools and scalable infrastructure to bring your AI ideas to life.

Key features:

- Serverless inference with leading LLMs and images AI integration
- Add workflows with knowledge bases for fine-tuned retrieval
- Build multi-agent systems that can work together
- Fully managed agents with built-in evaluation and traceability
- Guardrails for content moderation and sensitive data detection
- Embeddable chatbot snippets to easily integrate AI into your website
- Monitoring and rollback capabilities for safe experimentation

Ready to start building? Explore Gradient Platform for whatever you need to build, run, and scale AI agents.

About the author

Jean Silva
Content Marketing Manager

Jean Silva is a Content Marketing Manager at DigitalOcean. She has over 3 years of experience in product marketing, focusing on developer tools and cloud infrastructure. Outside of work, she enjoys traveling, photography, and hiking with her dog Max.

Related Resources

Articles
What is an Agent Harness? Architecture and Setup in 2026
Read time: 7

Articles
What is SGLang? 2026 Guide to the LLM Serving Framework
Read time: 9

Articles
DigitalOcean vs CloudNuke in 2026: AI Routing Comparison
Read time: 8

Start building today
Run AI agents, inference, and GPU workloads on infrastructure built for production.
Sign up

Company
About
Leadership
Blog
Careers
Customers
Partners
Referral Program
Affiliate Program
Press
Legal
Privacy Policy
Security
Investor Relations

Products
Gradient AI Agent
Knowledge Bases
Serverless Inference
Agent Routing
Inference Engine
Model Library
Droplets
GPU Droplets
Bare Metal GPUs
App Platform
Load Balancers
Managed Databases
Spaces
Block Storage
Network File Storage
Add-ons
uptime
Monitoring & Alerts
Reserved IPs
Identity and Access Management
Container Registry
View all Products

Resources
Community Home
Community Q&A
Tutorials
API Documentation
Customer Research
DigitalOcean Ventures
Open Source
Deployment Guides
Open Source
Newsletter Signup
Status
Pricing
Pricing Calculator
Migration Assistance
Release Notes
Code of Conduct
Web Hosting

Solutions
AI Strategy & ROI
GPU Infrastructure
Agentic Workloads
Web Hosting
Dev & Test
Game Servers
Website Hosting
Cloud Migration
Web Apps for Higher Ed
Nonprofit Hosting
Small Business
View all Solutions

Contact
Support
Sales
Report Abuse
System Status
Share your ideas

Developers
Docs
Get Started
Community
Webinars
API Reference
Release Notes
Video Tutorials
API Status
Open Source
Customer Stories
Partner Program
Marketplace
Hatch Startup Program
Shell
Terminal Server
Cloud Chats
Support Plans

© 2026 DigitalOcean, LLC. Sitemap